

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name : M Kavya

Reg. No : 24011101061

Experiment 4
**Comparative Study of Deep Convolutional Neural
Network Architectures Using Transfer Learning**

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

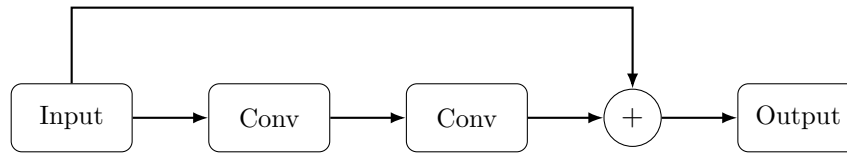


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution

- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

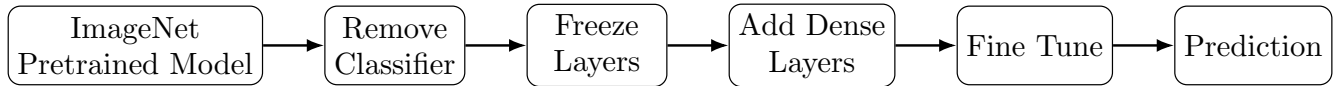


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Source Code

GitHub Repository

17. Hyperparameter Study

Compare the performance using different settings.

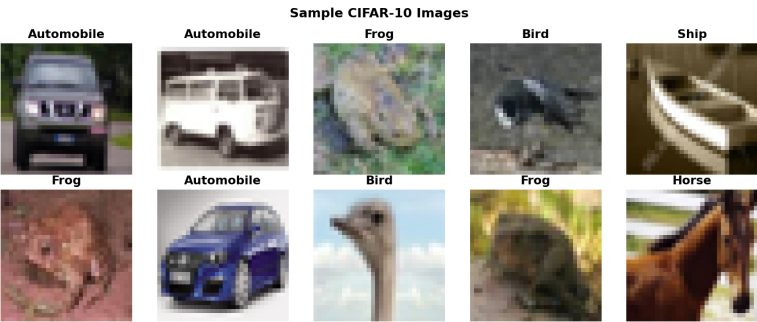
Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	32, 64
Epochs	5
Optimizer	Adam, SGD
Dense Units	128, 256

Each configuration below changes one hyperparameter at a time while the remaining values are held at the baseline (learning rate 0.001, batch size 32, 5 epochs, Adam, 128 dense units, convolutional base frozen). All configurations use VGG16 as the base and were trained for 5 epochs so that the effect of each hyperparameter could be observed quickly.

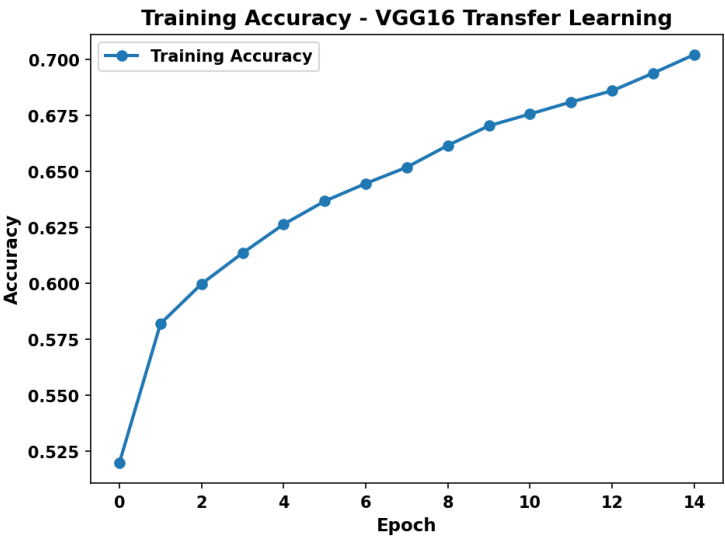
Configuration	Value	Validation Accuracy	Time (s)
Baseline	lr=0.001, bs=32, Adam, du=128	60.58%	71.2
Learning Rate	0.0001	57.64%	69.3
Batch Size	64	60.62%	56.8
Optimizer	SGD	55.06%	70.1
Dense Units	256	60.66%	69.7

Inference: Reducing the learning rate to 0.0001 and switching the optimizer to SGD caused the largest drops in validation accuracy (to 57.64% and 55.06% respectively), showing that the learning rate and optimizer choice matter the most for how quickly the classifier head converges in just 5 epochs. Batch size and the number of dense units had only a marginal effect (within about 0.1 percentage points of the baseline), so the model is fairly insensitive to those two settings in this range.

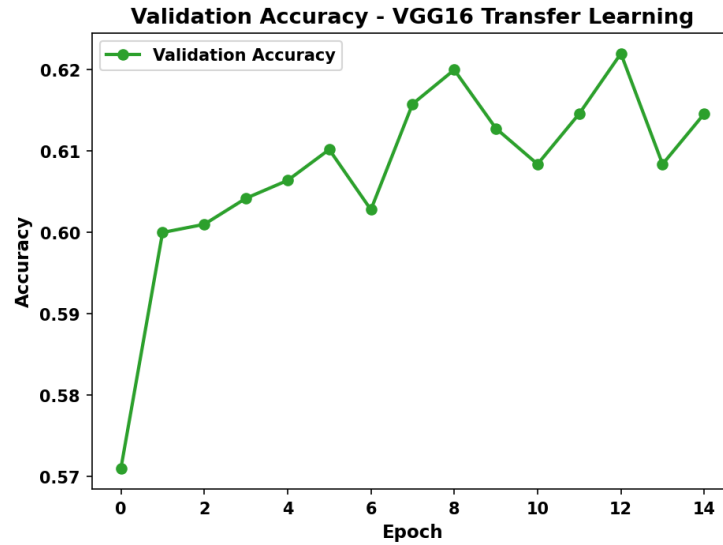
18. Plots



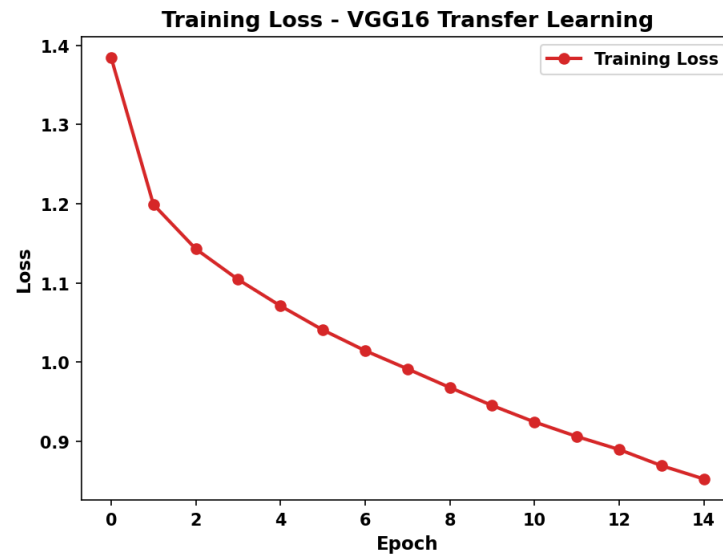
Inference: Ten CIFAR-10 sample images are shown. Similar classes such as Cat, Dog and Deer are harder to distinguish due to the low 32×32 resolution.



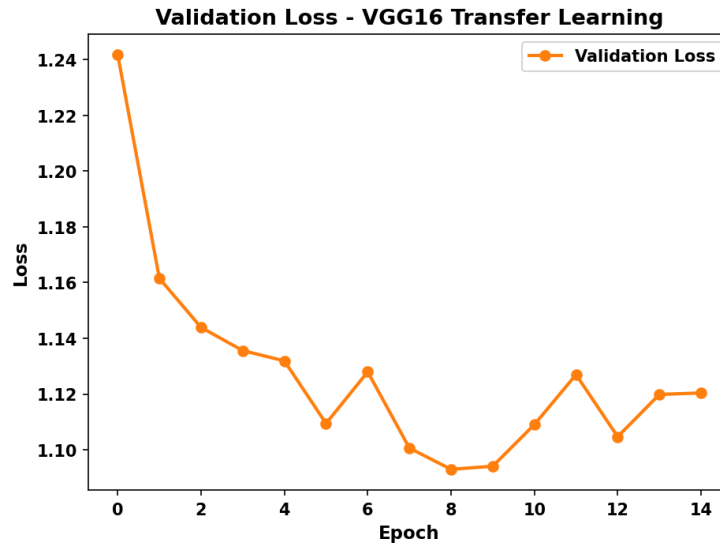
Inference: Training accuracy increases from about 52% to 70.2% over 15 epochs, showing steady learning by the classifier head.



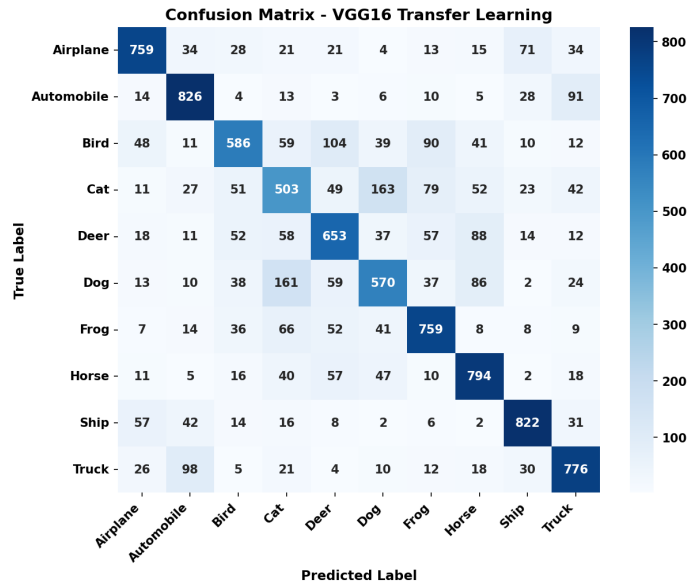
Inference: Validation accuracy reaches 61.46% and then levels off. The gap from training accuracy indicates slight overfitting.



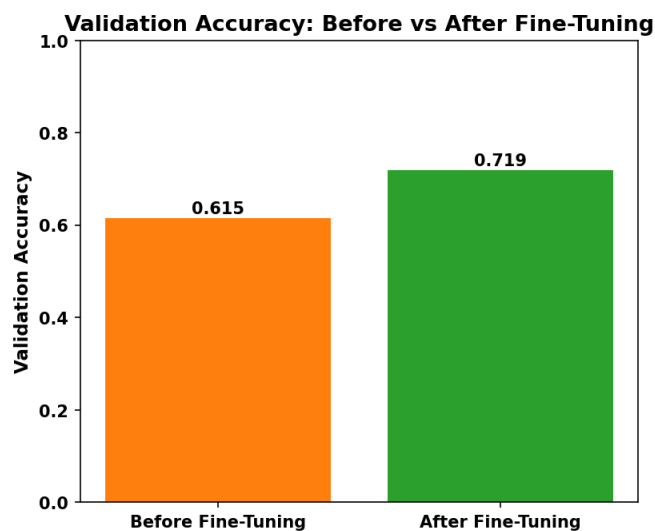
Inference: Training loss decreases steadily from 1.38 to 0.85, showing stable convergence with Adam.



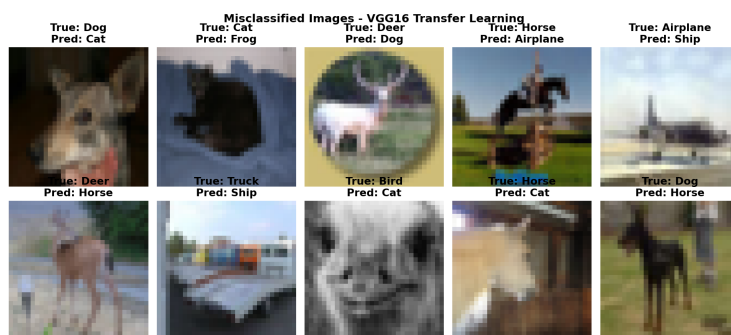
Inference: Validation loss decreases until about epoch 9 and then remains nearly constant, indicating limited improvement from the frozen base.



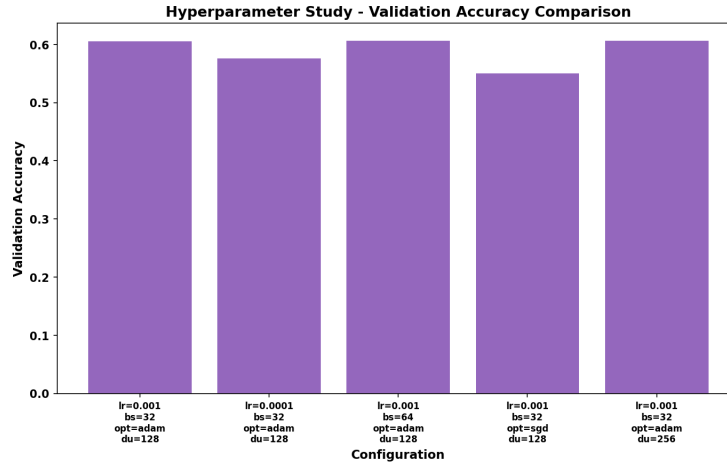
Inference: Ship, Automobile and Airplane are classified best, while Cat is most frequently confused with Dog and Bird.



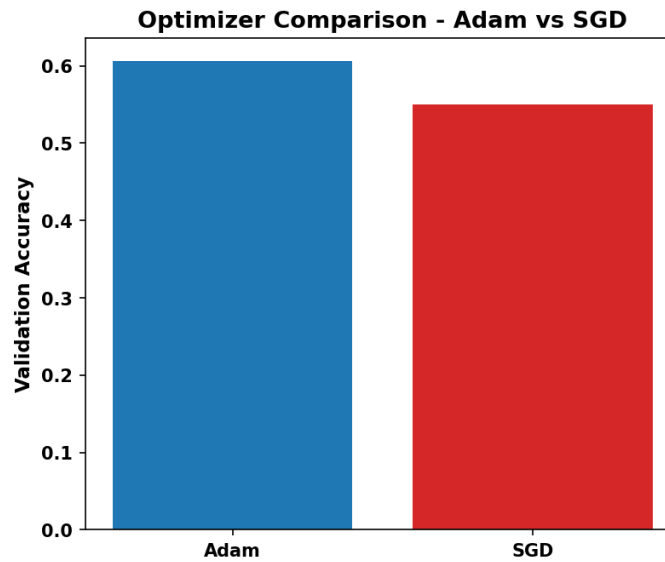
Inference: Fine tuning block5 improves validation accuracy from 61.46% to 71.88%, showing that adapting deeper features improves performance.



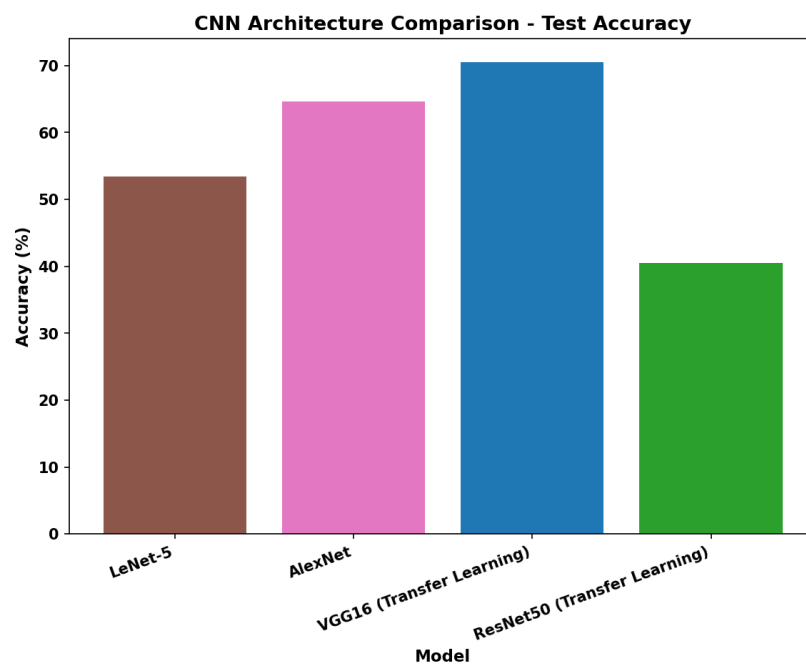
Inference: Most errors occur between similar classes, such as Cat–Dog and Deer–Horse, rather than unrelated classes.



Inference: The baseline, batch size and dense units give similar results, while a lower learning rate and SGD reduce validation accuracy.



Inference: Adam achieves 60.58% validation accuracy compared with 55.06% for SGD, showing better convergence under the same settings.



Inference: VGG16 achieves the highest accuracy at 70.48%, followed by AlexNet at 64.61%. ResNet50 performs lowest at 40.52% on CIFAR-10.

19. Results

19.1 Performance Metrics

Metric	Value
Training Accuracy	93.26%
Testing Accuracy	70.48%
Precision (macro)	70.28%
Recall (macro)	70.48%
F1-score (macro)	70.27%
Training Time	362.76 s (208.76 s main + 154.00 s fine-tuning)
Total Parameters	14,781,642

19.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5 (from scratch)	83,126	53.45	57.50 s
AlexNet (from scratch)	8,578,314	64.61	260.32 s
VGG16 (transfer learning)	14,781,642	70.48	362.76 s
ResNet50 (transfer learning)	23,851,274	40.52	153.99 s

20. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet won ImageNet 2012 with a large reduction in classification error. It introduced ReLU, Dropout and GPU training, making deep CNNs practical.

2. Why does VGG16 use only 3×3 convolution filters?

Stacked 3×3 filters provide a large receptive field with fewer parameters and add more nonlinearities, allowing deeper networks.

3. Explain the advantages of the Inception module.

It applies different filter sizes in parallel to capture multi-scale features. 1×1 convolutions also reduce computation and parameters.

4. What is the purpose of residual learning?

Residual learning uses skip connections to improve gradient flow and reduce the vanishing-gradient problem, allowing very deep networks to train effectively.

5. Differentiate LeNet and ResNet.

LeNet is a small 7-layer network designed for digit recognition, while ResNet50 is a deep 50-layer network using skip connections. ResNet is much larger and more powerful.

6. What is Transfer Learning?

Transfer learning uses a pretrained model and adapts it to a new task by replacing and training its classifier instead of training the entire network from scratch.

7. Why is fine tuning required?

Fine tuning adapts pretrained high-level features to the new dataset. In this experiment, it improved validation accuracy from 61.46% to 71.88%.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution increases the receptive field without increasing parameters, while transpose convolution increases the spatial size of feature maps through learnable upsampling.

9. Why do pretrained models converge faster?

Pretrained models already contain useful features such as edges and textures, so fewer parameters need to be learned from scratch.

10. Compare the computational complexity of LeNet and ResNet.

LeNet has far fewer parameters and requires less computation, while ResNet50 has millions of parameters and is much more computationally expensive.

21. Additional Exercises

1. Implement transfer learning using VGG16.

VGG16 was pretrained on ImageNet with its classifier replaced by new layers. After fine tuning block5, it achieved **70.48%** test accuracy.

2. Repeat the experiment using ResNet50.

ResNet50 achieved **40.52%** test accuracy. Its performance was lower because CIFAR-10's small 32×32 images are downsampled heavily.

3. Compare Adam and SGD optimizers.

Adam achieved **60.58%** validation accuracy, while SGD achieved **55.06%**. Adam converged faster in the same number of epochs.

4. Compare training with frozen layers and fine tuning.

Frozen layers achieved **61.46%** validation accuracy, while fine tuning increased it to **71.88%** by adapting deeper features.

5. Evaluate the model using Precision, Recall and F1-score.

The model achieved **70.28%** precision, **70.48%** recall and **70.27%** F1-score. Cat and Dog were the most difficult classes.

6. Compare the performance of LeNet, AlexNet and ResNet.

LeNet achieved **53.45%**, AlexNet **64.61%**, and ResNet50 **40.52%**. AlexNet performed best among these three in this experiment.

22. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>